```
        }
/*Opening a file(that is getting the file descriptor), whose
name was specified in command line argument 1 in read only mode
(specified by the symbolic constant O_RDONLY)*/
        sfd = open(argv[1],O_RDONLY);
        if (0 > sfd) {
                printf ("Cannot open File for reading \n");
                exit (1);
        }
/*Open the destination file in Write Only Mode(O_WRONLY), O_CREAT-
symbolic constant denotes that it will be created, O_TRUNC-data
will be truncated, if already existing, S_IRWXU-specifies the
permissions (read/write/excute for user)*/
        dfd = open(argv[2],O_WRONLY|O_CREAT|O_TRUNC,S_IRWXU);
        if (0 > dfd) {
                printf ("Cant open file for writing\n");
                exit (1);
        }
        while ((bytesread = read(sfd, buf, BLOCKSIZE))>0) {
                ret = write (dfd, buf, bytesread);
                if (-1 == ret) {
                        printf ("File Write Error\n");
                        exit (1);
                }
        }
        /*Closing the file descriptors explicitly*/
        close (sfd);
        close (dfd);
        return 0;
}
```

Now, note that here, the *open*, *read*, *write* and *close* functions have been used instead of *fopen*, *fgetc*, *fputc* and *fclose*. These actually translate to direct system calls. Instead of a file pointer from *fopen*, *open* returns a file descriptor number. The arguments of the *read* and *write* functions are of the general form: file descriptor, an array from where to write (in case of write) or into which to read data (in case of read), and third, the requested number of bytes to be read or to be written. The return values of *read* and *write* are the number of bytes successfully read or written. Here we do not have functions like *fgetc/fputc*, *fgets/fputs*, *fscanf/fprintf* or *fread/fwrite*, which can deal with characters, lines and formatted data and structures, respectively; *read* and *write* understand a file as a sequence of bytes only.

All standard library functions normally call the system calls internally; *fopen* should internally call *open*, or *fgetc* call *read*, etc. You can use *strace* (which traces system calls) to validate this. Compile *mycpy_lib.c and run* with source and destination file arguments, under *strace*, as follows:

```
bash-3.00$ strace ./a.out file1 file2

open ("file2", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
open ("file1", O_RDONLY)                = 4
```

The numbers on the right are the return values (in this case, file descriptors) of the *open* function, which was called internally by *fopen*.

Now, what about end-of-file handling? When there is no more data to be read, the *read* function returns a value of 0; this is received by *fgetc*, which then returns *EOF* (typically -1, but we should not assume a value, for portability) to the caller. Thus, the system need not store any special character to mark the end-of-file—just the number of bytes in the file.

So it is now clear that a file pointer is a higher-level abstraction than the file descriptor, and the end-of-file is not a special character stored in a file.

So what are the advantages and disadvantages of using system calls rather than library functions? The latter are obviously more portable; *fopen*, *fgetc*, *fputc*, *fclose*, etc, are supposed to be available on a wide variety of platforms, but system calls like *open*, *read*, *write* and *close* are available for a particular OS, say Linux. However, using system calls can make the program run faster on a specific OS.

In this article, I have briefly covered the basic C language file-system interface in Linux. Please read the manual pages to learn more functions like *link*, *symlink*, *unlink* *lseek*, *dup*, *access*, *stat*, *mkdir*, *rmdir*. Do also try implementing your own *ls* or *cat* command; that will help enhance your programming skills too. END

References

[1] Bach, Maurice J: 'The Design of the Unix Operating System', PHI
[2] Stevens, W Richard: 'Unix Network Programming: Interprocess Communication', Vol-2, 2nd Edition, Pearson Education
[3] Stevens, W Richard and Rago, Stephen A: 'Advanced Programming in the Unix Environment', 2nd Edition, Pearson Education
[4] Linux Manual Pages
[5] You can download the source code of this article from *http://www.linuxforu.com/article_source_code/july12/system_programming.zip*

Acknowledgement

I would like to thank Tanmoy Bandyopadhyay and Vikas Nagpal for their help in reviewing this article.

By: Swati Mukhopadhyay

The author is having more than 12 years of experience in academics and corporate training. Her interest areas are Digital logic, Computer architecture, Computer Networking, Data Structures, Linux and programming languages like C, C++. For any queries or feedback, she can be reached at *swati.mukerjee@gmail.com* or *http://swatimukhopadhyay.wordpress.com/*.